HOBIT

ARCHITECTURE AND PROTOCOLS

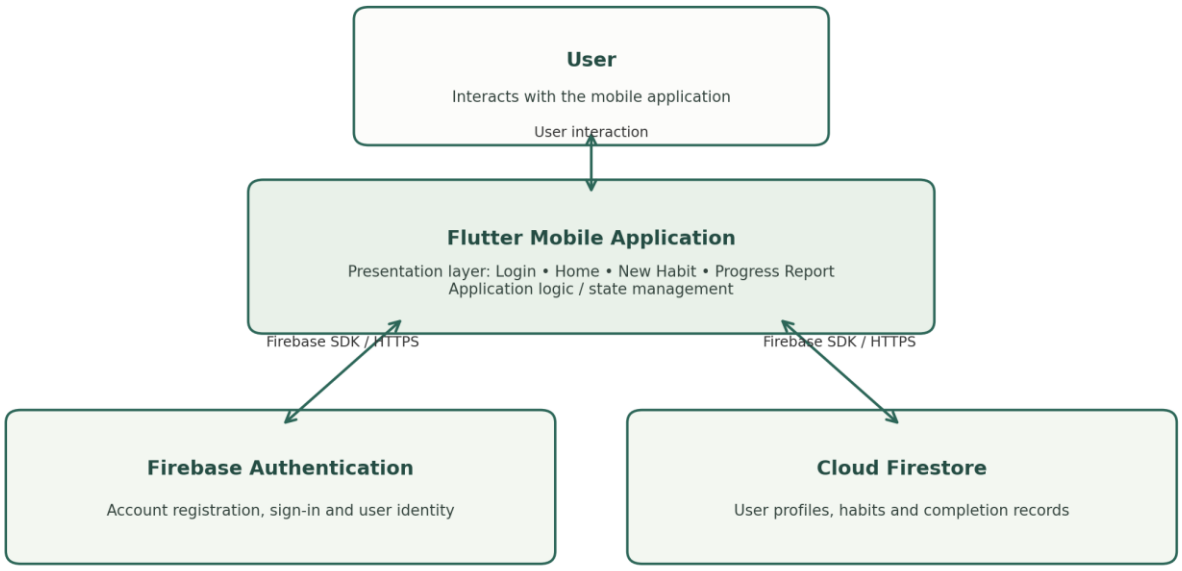
Mobile Application • Technical Design

1. System Overview

Hobit is a mobile habit-tracking application intended primarily for university students. It helps users create personal habits, define repetition and reminder preferences, mark habits as completed, and review their progress. The proposed solution uses Flutter for the mobile client and Firebase services for authentication and cloud data storage.

2. Proposed System Architecture

The system follows a client–backend architecture. The Flutter application provides the user interface and handles user interactions. Firebase Authentication manages account identity, while Cloud Firestore stores user-related application data. The client communicates with Firebase through the Firebase SDK over secure network connections.



Conceptual architecture — proposed design

3. Architecture Pattern

The application is planned to use a layered, MVVM-inspired structure to separate interface code from application logic and data access. This separation improves maintainability and makes individual parts easier to test.

Layer	Responsibility	Hobit example
Presentation / View	Displays screens and receives user input.	Login, Home, New Habit, Progress Report
ViewModel / Application logic	Validates input, coordinates actions and exposes UI state.	Sign-in flow, habit creation, completion updates
Repository / Data access	Encapsulates communication with Firebase services.	Authentication and Firestore operations

4. Technologies and Components

Technology / Component	Purpose
Flutter	Cross-platform mobile application framework and UI layer.
Dart	Primary programming language for the Flutter client.
Firebase Authentication	User registration, sign-in and identity management.
Cloud Firestore	Cloud-hosted NoSQL database for user and habit records.
Firebase SDK	Client-side integration with Firebase services.

5. Data Model

The following logical structure is proposed for Cloud Firestore. Each user's records are grouped under their authenticated user ID so that access can be restricted to the owner.

- users/{userId} — user profile information.
- users/{userId}/habits/{habitId} — habit name, start date, repetition settings, reminder preference and creation timestamp.
- users/{userId}/habits/{habitId}/completions/{dateId} — completion status and completion timestamp for a given date.

Exact field names and data types will be finalized during implementation. The structure above is a logical proposal rather than a claim that the database has already been implemented.

6. Communication Protocols

The Flutter client will use the official Firebase SDK to access Firebase Authentication and Cloud Firestore. Network communication with Firebase services is protected using HTTPS/TLS. The application should not transmit passwords or sensitive credentials through custom, unencrypted requests.

Authentication operations include account creation, sign-in and sign-out. Database operations include creating, reading, updating and deleting habit records, as well as recording completion events. Firestore listeners may be used when the interface needs to reflect database changes in real time.

7. Security and Privacy

- Require authentication for user-specific habit data.
- Configure Firestore Security Rules so a signed-in user can access only records associated with their own user ID.
- Validate user input before writing data to the database.
- Avoid storing plaintext passwords in the application database; authentication is delegated to Firebase Authentication.
- Request only the permissions required by implemented features, such as notification permission if reminders are enabled.

Security rules must be tested before deployment. Client-side checks alone are not sufficient to protect database records.

8. Main Application Workflows

User sign-in: The user enters credentials. Firebase Authentication verifies the credentials and returns an authenticated user identity. The app then opens the main screen.

Habit creation: The user enters a habit name, start date, repetition frequency and (if supported) a reminder time. The application validates the input and stores the habit under the authenticated user's record.

Habit completion: The user marks a habit as completed. The application records the completion for the relevant date in Firestore.

Progress review: The application reads the user's habits and completion records, then presents progress information in the Progress Report screen.

9. Interfaces and Error Handling

The main interfaces are the Flutter screens and the data-access methods that connect the application to Firebase. UI components should call application-logic methods rather than accessing database records directly. The data layer should return clear success or failure results so the interface can display appropriate feedback.

- Show a clear message when sign-in fails or the user has no network connection.
- Prevent saving a habit when required fields are missing or invalid.
- Handle loading, empty and error states on the Home and Progress Report screens.
- Do not display a successful save or completion state until the database operation succeeds.

10. Testing Considerations

- Authentication: valid credentials, invalid credentials and sign-out.
- Habit data: create, read, update and delete operations.
- Authorization: verify that users cannot read or modify another user's records.
- Input validation: empty habit name and invalid scheduling values.
- UI behavior: loading, empty, error and successful states across supported screen sizes.

11. Design Status and Assumptions

This document describes the proposed technical architecture for the milestone. Flutter, Firebase Authentication and Cloud Firestore are the selected technology direction. The exact MVVM implementation, Firestore schema, security rules, reminder mechanism and testing results must be confirmed during development. Push notifications (for example, Firebase Cloud Messaging) are not assumed to be implemented unless they are explicitly added to the project.

12. Conclusion

The proposed Flutter and Firebase architecture provides a practical foundation for Hobit's core features. Separating presentation, application logic and data access, together with authenticated access to user-specific Firestore records, is intended to support a maintainable and secure habit-tracking application.